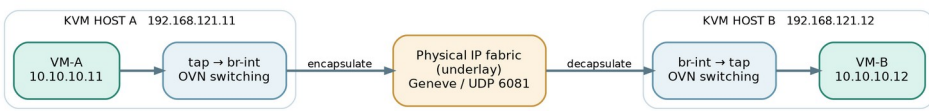
A VIRTUALIZATION-FIRST EXPLANATION

OVN NETWORKING FOR KVM

From a VM's virtual NIC to the physical wire

A concise, illustrated field guide for operators who know virtualization better than networking.



ONE LOGICAL NETWORK • TWO PHYSICAL KVM HOSTS

WHAT THIS BOOK DOES Builds the mental model first, then explains switching, routing, VLANs, tunnels, gateways and NAT through one running example.

Edition 1.4 · November 2026 · Generic KVM / libvirt / Open vSwitch / OVN

00 Before we begin

OVN is not a new hypervisor and not simply another Linux bridge. It is a **network control system** that tells Open vSwitch (OVS) on each KVM host how to implement logical switches, routers and policy. A VM still has a virtio NIC and a tap interface; OVN changes what the host does with its traffic. [1, 7, 8]

Read this as a virtualization operator

Familiar idea	OVN / OVS equivalent
VM NIC / tap	The VM-facing interface on a KVM host
Port group / virtual network	OVN logical switch (approximate analogy)
Virtual NIC connection	OVN logical switch port + OVS interface binding
Virtual switch dataplane	Open vSwitch (usually br-int)
Distributed virtual router	OVN logical router
Edge / uplink service node	OVN gateway chassis + physical uplink

The running example: two hosts, kvm-a and kvm-b, run VMs in 10.10.10.0/24. Their transport IPs are 192.168.121.11 and 192.168.121.12. Later we add a second subnet and a VLAN-backed provider network 192.168.122.0/24. These are example-only addresses; no OpenStack components are assumed.

Contents

01 The three pictures to keep separate

Physical-to-logical architecture foldout

02 The OVN components

03 How a VM joins a logical switch

04 Switching between KVM hosts

05 Routing without a router VM

06 How physical VLANs connect

07 Gateways, SNAT and DNAT

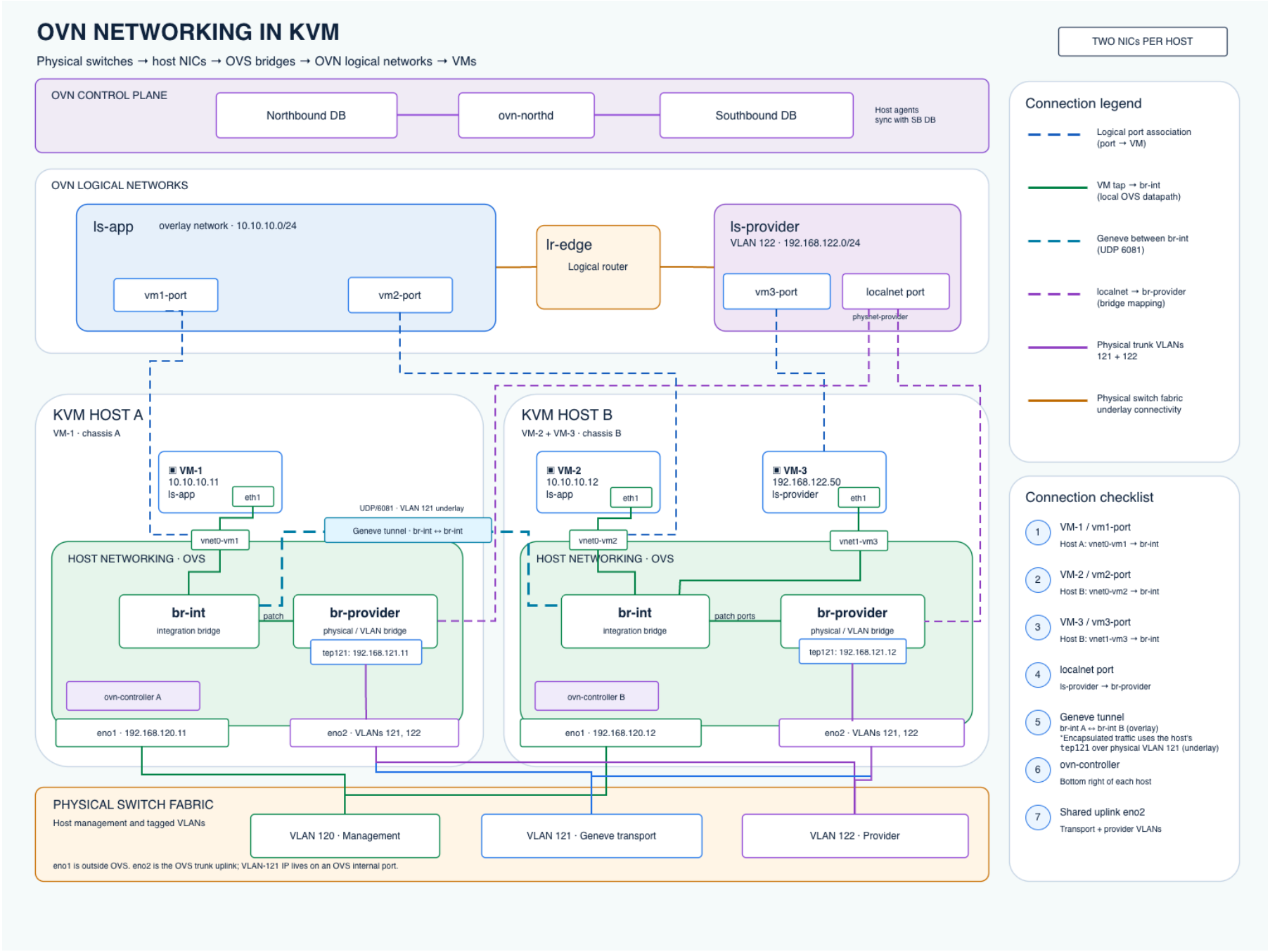
08 Availability and a 100-host design

09 What to check when traffic fails

10 Essential rules and practical commands

12 Glossary / 13 Sources and next steps

PHYSICAL-TO-LOGICAL ARCHITECTURE | GENERIC KVM + OVS + OVN

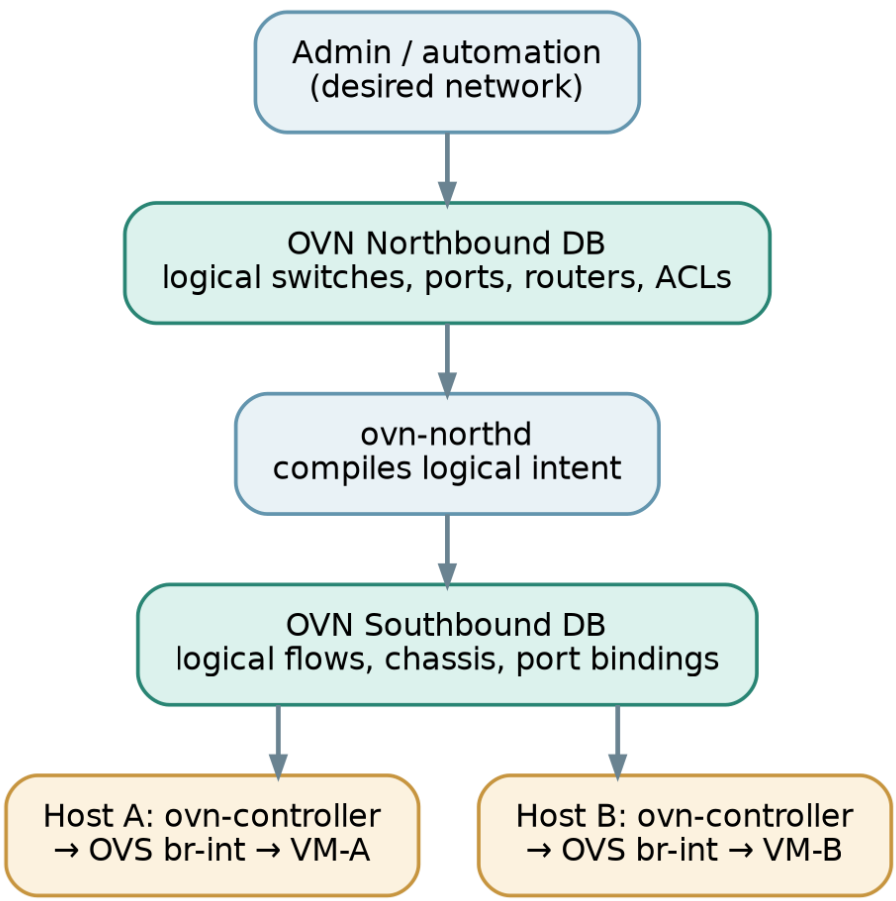


Physical IPs are underlay addresses; OVN logical switches and VM IPs form the guest network. Diagram is conceptual, not a literal packet trace.

01 The three pictures to keep separate

Think of an OVN environment as three related diagrams, each answering a different question:

- **The logical topology:** Which VMs are connected to which switches and routers?
- **The physical topology:** Which KVM hosts, NICs, VLANs, leaf switches and IP routes carry the packets?
- **The control path:** Who translates desired logical topology into forwarding rules on each host?



Control information flows downward; packet forwarding happens locally on hosts.

A logical switch is **not** an OVS bridge with the same name. A logical router is **not** a Linux network namespace or a dedicated VM. OVN represents both as distributed logical datapaths, realized in OVS flows on the participating chassis. [1]

MENTAL MODEL The logical diagram tells you what connectivity should exist. The physical diagram tells you how packets reach the destination host. OVN's control plane joins the two.

02 The OVN components

Control plane: describes and distributes intent

- **Northbound DB (NB):** desired logical switches, ports, routers, ACLs and NAT rules. An operator or automation writes here.
- **ovn-northd:** translates the logical configuration into lower-level logical flows and port-binding information.
- **Southbound DB (SB):** the shared view of logical flows, chassis identities, tunnel endpoints and where VM ports currently live.
- **ovn-controller (each host):** reads SB and programs the local OVS dataplane; reports the chassis and bound VIFs.

Dataplane: actually forwards packets

Open vSwitch (OVS) is the software switch on each host, typically with an OVN integration bridge named `br-int`. It handles locally attached tap ports, applies OVN-programmed flows and sends packets through tunnels or physical uplinks. The VM never communicates with the NB or SB database itself. [1]

Chassis means an OVN transport node: usually a KVM compute host or a gateway host. The controllers and databases can run on other machines. A central controller is not in the normal per-packet path.

AVAILABILITY IMPLICATION If the control plane becomes temporarily unavailable, established OVS forwarding normally continues with the programmed state, but new bindings and network changes may not propagate.

03 How a VM joins a logical switch

A KVM guest has a virtio NIC. Libvirt commonly creates a tap device (`vnetN`) and attaches it to the OVS integration bridge. OVN needs to know which **logical switch port** that tap represents. [1, 7, 8]

For example, `VM-A` has MAC `02:ac:10:00:00:11`, IP `10.10.10.11`, and is attached to logical switch `ls-app`.

```
# Desired logical network (on an OVN admin host)
ovn-nbctl ls-add ls-app
ovn-nbctl lsp-add ls-app vm-a-port
ovn-nbctl lsp-set-addresses vm-a-port \
  "02:ac:10:00:00:11 10.10.10.11"
```

On the KVM host, the OVS Interface representing the VM must have `external_ids:iface-id=vm-a-port`. An integration layer (libvirt hook or orchestrator) normally ensures that the guest interface and the OVN port have matching IDs.

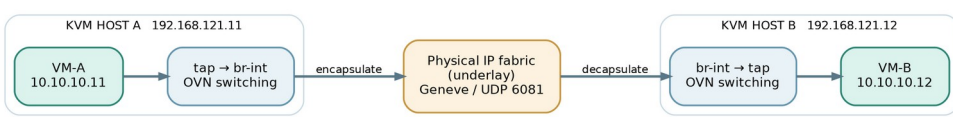
```
# Conceptual manual binding when vnet0 already exists
ovs-vsctl add-port br-int vnet0 -- \
  set Interface vnet0 external_ids:iface-id=vm-a-port
```

If libvirt itself attaches the VM to OVS, do not add the same tap a second time. Libvirt's Open vSwitch `virtualport` uses an `interfaceid` UUID; make the OVN logical port name that same UUID, or implement a consistent mapping in your integration code. [7, 8]

IMPORTANT Setting an IP in `lsp-set-addresses` records the port's known addresses (not its port-security policy); it does not configure the guest operating system. Use static IP, guest DHCP or OVN DHCP options separately.

04 Switching between KVM hosts

Put VM-A (`10.10.10.11`) on `kvm-a` and VM-B (`10.10.10.12`) on `kvm-b`. Both join `ls-app`, so they behave as if plugged into one Layer-2 switch—even though they run on separate physical machines.



Overlay example: tenant Ethernet frames are carried across an ordinary IP underlay.

OVN learns that `vm-b-port` is bound to `kvm-b`. On `kvm-a`, OVS processes the logical switching pipeline, wraps the frame in **Geneve** and sends it to the other host's transport IP. OVS on `kvm-b` removes the encapsulation and delivers the original frame to VM-B. [1, 5]

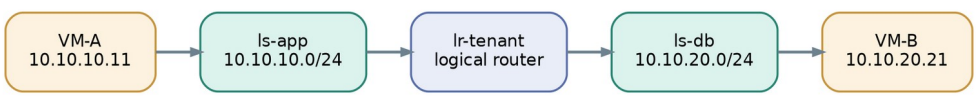
What the physical fabric sees

The physical switches see traffic **between chassis IP addresses**, not the tenant IPs directly. They need IP reachability between `192.168.121.11` and `192.168.121.12`, an appropriate MTU and permission for Geneve (normally UDP 6081). They do not need a separate VLAN for every logical network.

If both VMs happen to be on the same host, OVS can deliver the frame locally: no transport tunnel is needed for that hop. A VM migration changes the port binding; the logical address and network do not have to change.

05 Routing without a router VM

Now add `ls-db` (`10.10.20.0/24`) and put VM-C (`10.10.20.21`) on it. Create one logical router, `lr-tenant`, with interfaces `10.10.10.1/24` and `10.10.20.1/24`. VM-A uses `10.10.10.1` as its default gateway. [1, 2]



The logical view: routing exists, but there is no router appliance in the middle.

When VM-A sends to VM-C, local OVS can apply both the logical switch and router processing. If VM-C is remote, the resulting packet travels to its host. There is usually **no compulsory detour** through a central gateway to route between internal OVN logical switches.

The OVN router does ordinary L3 work: picks the next hop, decrements TTL and rewrites the Ethernet header. OVN distributes that behavior across the chassis where the relevant VMs run. [2]

DO NOT CONFUSE Distributed router = routing logic available on participating hosts. Gateway chassis = a specific host chosen for external-network functions that cannot safely be fully distributed.

06 How physical VLANs connect

A VM sometimes needs to be on a real Ethernet VLAN—for example, to reach a NAS, legacy VMs or bare-metal systems without routing. OVN uses a **localnet logical switch port** to join a logical switch to an existing physical L2 network. [1, 4, 5]

Think “bridge mapping”, not “tunnel”

A host has an OVS provider bridge (`br-provider`) with a physical NIC or bond attached. The host advertises that `physnet` maps to that bridge. OVN then links the relevant logical switch’s `localnet` port to the mapped provider bridge (via OVS patch connectivity).

```
# Chassis-local mapping (illustrative)
ovs-vsctl set Open_vSwitch . \
  external_ids:ovn-bridge-mappings=physnet:br-provider

# OVN logical connection to that mapped network
ovn-nbctl ls-add ls-vlan122
ovn-nbctl lsp-add ls-vlan122 ln-vlan122
ovn-nbctl lsp-set-type ln-vlan122 localnet
ovn-nbctl lsp-set-addresses ln-vlan122 unknown
ovn-nbctl lsp-set-options ln-vlan122 network_name=physnet
ovn-nbctl set Logical_Switch_Port ln-vlan122 tag_request=122
```

Set the appropriate VLAN tag (for example 122) on the localnet port, and make the physical switch carry VLAN 122 to the host. OVN/OVS handle the tagging at the boundary; the guest can see an untagged vNIC. The physical VLAN becomes part of the datapath.

AVOID AN EASY MISTAKE `br-provider` does not need the SNAT address as a Linux interface IP. A router gateway address and its NAT address are OVN logical configuration; the OVS provider bridge is primarily a Layer-2 attachment.

07 Gateways, SNAT and DNAT

A private logical network can reach external networks through a logical router connected to an external logical switch, whose localnet port reaches a real network. The router has a port on the external subnet; a default route points to the upstream physical router. [1, 3, 4]



North–south path. Shared SNAT state is handled on an active gateway chassis.

Example: one upstream-facing source address for many VMs

VM-A (10.10.10.11) sends toward an upstream network. OVN applies SNAT to 192.168.122.20; the physical gateway 192.168.122.1 returns replies. The external logical router port could use 192.168.122.10/24. Internet access would require further NAT upstream, as these are private example addresses.

Shared, one-to-many SNAT usually needs one active gateway chassis for the relevant gateway port: many connections share the same external identity and need consistent state and a single physical place to own the gateway MAC/IP. OVN redirects those packets to the selected chassis. [1]

DNAT maps an external address/port to an internal VM. A suitable one-to-one NAT mapping can be processed on the VM's chassis in a distributed design, provided the external path and OVN configuration support it; do not assume all north–south traffic must traverse one gateway.

Who does what?

- **Logical router port:** owns an external-side MAC and IP; is not a Linux IP on `br-provider`.
- **localnet port:** joins `ls-external` to a mapped physical bridge / VLAN.
- **Gateway chassis:** hosts the active physical-facing router functions for a distributed gateway port.
- **Physical router:** routes onward to WAN, core or other networks.

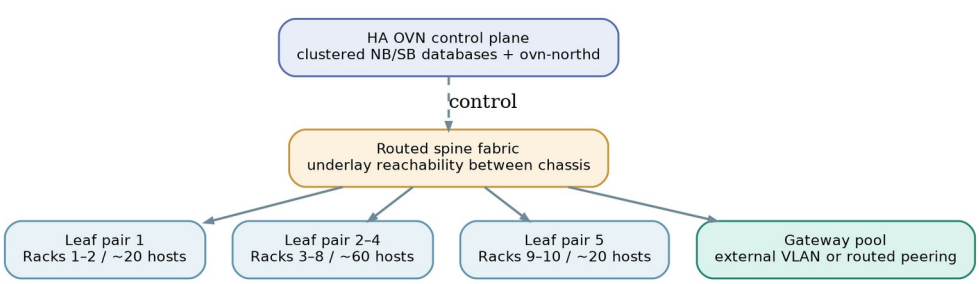
08 Availability and a 100-host design

OVN separates two availability questions: **Is the desired configuration still being distributed?** and **Can traffic still be forwarded when something fails?** Keep the answer to each explicit. [1, 3]

Small cluster

In a two-host lab, the KVM hosts can also be eligible gateways, assuming both have access to the external VLAN. Configure more than one gateway chassis in priority order. OVN selects one active chassis for a distributed gateway port and can fail over on failure. Multiple logical routers can have different active gateways, spreading aggregate load.

A larger example: 100 hosts / 10 racks



Illustrative leaf/spine layout: two racks per leaf pair and distinct gateway capacity.

- Use a **routed IP fabric** for chassis/tunnel endpoint reachability; logical tenant networks need not be stretched as VLANs.
- Run redundant **NB/SB database services and northd** on stable control nodes; place replicas across failure domains.
- Deploy a **gateway pool** with external L2 attachment or an explicitly designed routed handoff; size for throughput and failover capacity.
- Distribute different tenant routers across active gateways, but do not expect one shared-SNAT flow to be load-balanced across all gateways by default.
- If tenants use direct physical VLANs, plan VLAN reachability and bridge mappings on every chassis that must host those VMs.

Newer OVN dynamic-routing integrations can exchange routes with an external routing daemon such as FRR. That is an advanced alternative to a single shared L2 gateway segment—not a prerequisite for learning OVN. [9]

09 What to check when traffic fails

Troubleshoot in the same order that a packet travels. This makes it possible to distinguish a guest problem, a logical-config problem and a physical transport problem.

Question	Useful verification
Does the VM have the right vNIC and IP?	virsh domiflist <vm>; ip addr and ip route inside guest
Is its tap on OVS and mapped?	ovs-vsctl show; ovs-vsctl list Interface
Does the logical network exist?	ovn-nbctl show; ovn-nbctl lsp-get-up <port>
Where is the port bound?	ovn-sbctl show; inspect Port_Binding
Does OVN route as expected?	ovn-trace (logical packet simulation)
Can the physical hosts reach each other?	ping / route to chassis IPs; underlay MTU
Are encapsulated packets visible?	tcpdump -ni <nic> udp port 6081
Can the physical VLAN be reached?	bridge mapping, port tags, switch trunk, tcpdump
Is the gateway on the right chassis?	ovn-nbctl lrp-get-gateway-chassis <lrp>

`ovn-trace` tests the **logical** pipeline; it does not prove that a physical uplink, VLAN trunk, MTU or tunnel endpoint is functioning. Conversely, physical pings between KVM hosts do not prove that a VM port is bound to the correct logical switch. [2]

DIAGNOSTIC HABIT Always identify the flow as local L2, cross-host L2, east–west L3, localnet/VLAN, or north–south NAT before following the packet.

10 Five rules to remember

- **OVS forwards; OVN decides the intended forwarding.** They are related, not interchangeable.
- **A VM connects to `br-int`, but belongs to an OVN logical switch through its port binding.**
- **A logical switch does not imply a physical VLAN.** In overlay mode, the physical fabric transports tunnel packets between chassis IPs.
- **A logical router is normally distributed.** Routing between internal logical networks does not require a central router VM.
- **The external boundary is different.** A localnet port bridges into a physical L2 network; shared SNAT typically uses one active gateway chassis per relevant gateway port.

Two designs at a glance

Overlay-first design	VLAN-first design
Tenant L2 is carried by Geneve between KVM hosts.	Tenant L2 is carried by configured physical VLANs.
Physical fabric needs routed transport reachability.	Physical fabric must carry each used VLAN.
More flexibility for tenant isolation and addressing.	Simple connection to existing L2, but dependent on VLAN availability.
External access normally goes through localnet gateway design.	External routing can be performed by the physical fabric.

Caution on "VLAN-only": attaching two hosts to the same localnet/VLAN avoids a Geneve tunnel for that L2 path. It does **not** automatically mean that all OVN logical routing between different VLAN-backed networks is tunnel-free; OVN may still use tunnels for distributed routing. For a strictly tunnel-free design, decide deliberately where inter-VLAN routing should happen. [1]

11 A compact lab recipe

This is an **illustrative sequence**, not a complete production deployment or a copy-paste installer. Assume OVN NB/SB and `ovn-northd` are running, each host has OVS and `ovn-controller`, and the hosts already have transport IP reachability.

First: one isolated logical switch

```
ovn-nbctl ls-add ls-lab
ovn-nbctl lsp-add ls-lab vm-a-port
ovn-nbctl lsp-set-addresses vm-a-port \
  "02:ac:10:00:00:11 10.10.10.11"
ovn-nbctl lsp-add ls-lab vm-b-port
ovn-nbctl lsp-set-addresses vm-b-port \
  "02:ac:10:00:00:12 10.10.10.12"
ovn-nbctl show
```

Plug the two VM tap devices into `br-int` with matching `iface-id`s, using a libvirt integration or a deliberate host-side script. Check the SB port bindings and test a VM-to-VM ping.

Then: add a router and another network

```
ovn-nbctl ls-add ls-db
ovn-nbctl lr-add lr-lab
ovn-nbctl lrp-add lr-lab lrp-app \
  02:ac:ff:00:00:01 10.10.10.1/24
ovn-nbctl lsp-add ls-lab app-to-router
ovn-nbctl lsp-set-type app-to-router router
ovn-nbctl lsp-set-addresses app-to-router router
ovn-nbctl lsp-set-options app-to-router \
  router-port=lrp-app
```

Repeat the router-port/peer-switch-port pattern for `ls-db` with gateway `10.10.20.1/24`. Finally, configure each guest default route. For a full working command sequence and `ovn-trace` example, use the official OVN sandbox tutorial. [2]

SAFETY Do not move a production management NIC into an OVS bridge while connected remotely without a recovery path. Keep management and packet-transport configuration explicit.

12 Glossary

Term	Meaning in this guide
Underlay	The real IP/VLAN fabric between KVM hosts.
Overlay	The logical Ethernet network transported in tunnels.
br-int	The OVS integration bridge on a chassis.
VIF / tap	The host-side connection for a VM virtual NIC.
Chassis	An OVN transport host, compute or gateway.
Port binding	OVN's record of which chassis hosts a logical port.

Term	Meaning in this guide
localnet	An OVN port connecting logical L2 to existing physical L2.
Gateway chassis	Selected host for physical-facing gateway functions.
SNAT / DNAT	Source / destination IP address translation.

13 Sources and next steps

The explanations are based on OVN's own architecture and database documentation, with libvirt and OVS references for the VM attachment. All links below go to official project documentation.

Primary official documentation

[1] OVN Architecture

[2] OVN Sandbox Tutorial

[3] OVN Gateway High Availability

[4] OVN Northbound schema

[5] OVN Southbound schema

[6] OpenStack Networking introduction (concepts only)

[7] libvirt Domain XML

[8] Open vSwitch with libvirt

[9] OVN Dynamic Routing architecture

The OpenStack Networking introduction is used only for general network, port, provider-network and routing terminology. This guide does not assume Neutron, Nova, ML2 or any other OpenStack component.

Three useful lab exercises

- Move VM-B from `kvm-b` to `kvm-a`. Confirm that its logical port changes chassis while its IP stays the same.
- Add a second logical switch and route between two VM subnets. Use `ovn-trace` to explain each hop.
- Attach an external `localnet` and configure shared SNAT. Check that return traffic reaches the active gateway chassis.

End of guide · Keep the logical, physical and control-plane pictures separate.